

2. 데이터 와 엔티티

본 시스템이 무엇을 데이터로 보는가 를 한 곳에 정리. TypeDB 안의 5종 엔티티와 총 38개 노드의 의미.

📖 용어 풀이 (자료 곳곳에서 등장)

약어 / 용어	풀이
EMD_	읍·면·동 (Eup-Myeon-Dong). 한국 행정구역 표준 약어. 예: EMD_여수_상암동 = 여수시 상암동
S = 0.534	s-priority 의 약자. 종합 위험 점수 (0 ~ 1 범위)
TypeQL	TypeDB 의 쿼리 언어. SQL 이 관계형 DB 의 쿼리 언어인 것과 같은 관계
advisory	MOCK 데이터 인용 시 LLM 답변에 자동으로 따라붙는 "추정값입니다" 같은 경고문
fun	TypeDB 3.0 의 함수 기능. 판단 로직을 DB 안에 저장 (자세한 건 [3_추론_과_LLM.md] § 1)

1. TypeDB 안의 5종 엔티티 — 한 장 시각화



29 + 5 + 2 + 1 + 1 = 38개 노드

2. 각 엔티티의 비유와 역할

엔티티	비유	역할
 source	원산지 표기	데이터별 신뢰도(REAL/MOCK/EXCLUDE) 기록
 segment	환자 종합 차트	입력 + 추론 결과(write-back) 한 노드에 통합
 threshold-set	합격 커트라인	점수를 대응 단계(1~5)로 변환하는 기준
 weight-set	과목별 가중치	여러 사실 데이터를 점수로 합치는 배합비
 run-context	디지털 영수증	"왜 이 결과가 나왔지?" 사후 추적의 출발점

3. source — 29건의 데이터 카탈로그

source 는 데이터 자체가 아니라 "데이터 종류 1개의 메타정보" 다.

```
 source 노드 1개 = "데이터 종류의 카탈로그"
source-id           "SOURCE_KMA_OBSERVED_WEATHER"
source-name         "기상 관측/강수 관측"
availability-status "REAL"
```

비유: 마트 영수증이 아니라 장보기 리스트의 한 줄.

3종 status 의 의미

 REAL	"진짜 받을 수 있다"	18건	공공데이터/국가 시스템
 MOCK	"지금은 가짜로 채움"	6건	담당 부서 시스템 미연계 → LLM 인용 시 "추정값" advisory 자동 부착
 EXCLUDE	"이번 스터디 범위 밖"	5건	대체 가능 / 보안 / 범위 밖 → 의사결정에 영향 없음
Σ 29			

MOCK 6건의 정체 (공통점: 현장 운영 데이터)

- 1. 도로 진입 가능성 / 2. 소화전·비상소화장치 / 3. 지표연료 상태 (낙엽·마른풀) /
 - 2. 작업장 위험 / 5. 소방 자원 가용성 / 6. 예비주수 장비 능력
- 운영 전환 체크리스트: 이 6건을 실제 source 로 갈아끼우면 운영 단계로 전환.

4. segment — 종합 차트 (입력 + 추론 결과)

segment 5건이 적재돼 있다. 각 노드는 **입력 attribute** 와 **추론 결과 attribute (write-back)** 를 같이 가진다.

[입력]

segment-id, segment-name, admin-region
risk-grade, alert-level
residential-population, forest-distance-m
wind-speed, safety-class

[추론 결과 — write-back]

s-priority (점수, 0~1)
base-state (점수만으로 결정한 단계)
state (override 적용 후 최종 단계)
state-override-reason (왜 override 됐는지)

5건 segment 의 추론 결과

동네	점수	base-state	최종 state	비고
여수 상암동	0.534	Review	Priority ↑	경보 override
장흥 유치면	0.454	Review	NotActionable 🛑	안전 차단
광주 동구 (사례)	0.180	General	General	변화 없음
무안 운남면	0.295	Enhanced	Enhanced	변화 없음
나주 봉황면 ★	0.350	Enhanced	Enhanced	v2 정책이면 격상

5. threshold-set — 정책 카드 (코드 0줄 시뮬레이션의 핵심)

 threshold-set v1 (현행)

th-low0.20

th-mid-low0.40

th-mid-high0.60

th-high0.80

v2 (보수적)

0.15

0.30

0.50

0.70

v2 가 "보수적"인 이유: 경계선이 왼쪽 이동 → 동일 점수가 더 높은 단계로 격상됨.

v1 → v2 시뮬레이션 결과 (실제 적재 기준)

동네	점수	v1 정책	v2 정책	결과
여수 상암동	0.534	Review	Priority	격상 (경보 override 유지)
★ 나주 봉황면	0.350	Enhanced	Review	한 단계 격상! ↑
무안 운남면	0.295	Enhanced	Enhanced	변화 없음

🎯 **포인트:** 코드를 한 줄도 안 고치고 INSERT 한 줄로 정책 v2 를 추가했더니, 나주 봉황면이 즉시 격상 됨. 데이터가 곧 정책 인 구조의 실증.

6. State 6종 — 산불 대응 단계 사전

점수 낮음 → 점수 높음		
① GeneralManagement	평상 관리	● 일상 순찰
② EnhancedMonitoring	감시 강화	● 모니터링 ↑
③ ReviewPreWatering	예비 주수 검토	● 회의 소집
④ PriorityPreWatering	우선 예비 주수	● 자원 배정
⑤ ImmediatePreWatering	즉시 예비 주수	🔥 출동·살수
□ NotActionable	작업 불가	🚫 점수 무관, 안전 강제 차단

예비 주수(): 산불 발생 전 위험 지역에 미리 물을 뿌리는 예방 조치. 의사결정의 핵심 액션.

7. 왜 이 5종 구조인가

이 구조의 핵심은 "데이터가 추론을 낳고, 추론이 다시 데이터가 되어(write-back), 모든 과정이 영수증(run-context)으로 남는" 투명한 의사결정 체계다.

특히 threshold-set 과 weight-set 을 **별도 노드** 로 분리한 게 결정적. 이게 코드 안에 상수로 박혀 있었다면, 정책을 바꿀 때 코드 배포가 필요했을 것이다. 노드로 빠져 있기 때문에 INSERT 한 줄로 정책 시뮬레이션이 가능하다.



한 줄 요약

"source(원료) + weight·threshold(레시피) → segment(요리) → run-context(영수증). 레시피를 노드로 빼낸 게 코드 0줄 시뮬레이션의 핵심."